Rule Set Interpretability with Large Language Models

Francisca Roque
96814
Instituto Superior Técnico
Lisbon, Portugal

Dinis Corte Real
100298
Instituto Superior Técnico
Lisbon, Portugal

Javier Liébana
Feedzai
Madrid, Spain

Jean Alves
Feedzai
Lisbon, Portugal

ACM Reference Format:
Francisca Roque, Dinis Corte Real, Javier Liébana, and Jean Alves. 2025. Rule Set Interpretability with Large Language Models. In *Proceedings of (Applications of Data Science and Engineering).*

1 Introduction

Feedzai, a company specialized in financial fraud detection, utilizes advanced machine learning models to identify fraud patterns in banking transactions. Financial institutions require transparency in model decisions to build stakeholder trust and comply with regulations. These decisions are based on rule sets, with each rule consisting of a set of conjunctions that compare variables with specific thresholds.

Rule sets offer an interpretable solution, but as they become more complex, they may contain hundreds of highly complex rules with an immense number of conditions, making even rule based models challenging to understand.

To address this challenge, this project focuses on enhancing the interpretability of complex rule sets by leveraging Large Language Models (LLMs) [15] to generate a quick and easy to digest description and name for the rule. These rule sets, generated by rule induction algorithms such as RIFF [11], are built using "if-then" logic but can grow too intricate for straightforward interpretation. An example of a rule, along with its respective description and name, is presented below:

Rule: `housing_status_BA` > `0.5` ∧ `device_os_windows` > `0.5` ∧ `phone_home_valid` <= `0.5` ∧ `bank_branch_count_8w` <= `2.5` ∧ `name_email_similarity` <= `0.2082037702202797`

Description: The rule is checking if the house status is "BA", if the device uses the operative system Windows, if the home phone number is invalid, if the bank branch count is 2 or less, and if the person name and email name similarity is low, with a value equal or below the threshold of 0.2082037702202797.

Name: BA Housing with inconsistent data.

By making these rule sets more accessible, we aim to improve transparency, build trust among stakeholders, and facilitate the adoption of fraud detection systems in high-stakes, regulated environments.

To maximize the effectiveness of the LLMs, we explored various prompt engineering techniques, in order to guide the models in producing relevant and accurate rule descriptions. The process began with simpler prompting methods and gradually increased in complexity, ultimately focusing on selecting an optimal example based on rule similarity. We then benchmarked the performance of different LLMs to evaluate how well they could generate human readable descriptions, using ROUGE, a semantic similarity metric like [10] to measure the alignment of the generated descriptions with human generated descriptions.

This report is structured as follows: First, in the "State of the Art" Section the key concepts necessary for understanding the project are outlined. Next, in Section "Methodology", we detail how we generate a dataset with rules, their corresponding descriptions, and their names; followed by a detailed description of the prompting strategies to be tested, and, additionally, the implementation of a metric to evaluate the quality of LLM generated rule descripitons. Each methodological subsection is paired with a corresponding results subsection, where the findings are presented and analyzed. Finally, a discussion of the overall conclusions and their implications are presented, as well as potential directions for future work.

Applications of Data Science and Engineering, IST
© 2025

2 State of the art

2.1 RIFF

Traditional rule-based systems are valued for their interpretability but often demand significant manual effort for creation and maintenance. RIFF [11] is an algorithm designed to generate interpretable rule sets optimized for low false positive rates (FPR) across various domains. By utilizing decision trees, RIFF enhances predictive performance and simplifies rule generation. In this work, RIFF was employed to generate a set of rules that were used as a benchmark for evaluating the outputs of LLMs.

2.2 Large Language Models (LLMs)

LLMs represent a transformative advancement in natural language processing (NLP) [2], enabling systems to perform tasks such as language translation, summarization, and reasoning with high accuracy [15]. Built upon transformer-based neural networks [1], these models leverage self supervised learning [6] from vast datasets, capturing intricate linguistic structures and contextual relationships.

One of the standout capabilities of LLMs is their proficiency in generating natural language text that is not only syntactically correct but also contextually relevant and coherent [7] . This makes them invaluable for tasks requiring fluent language generation, such as creating descriptions, crafting summaries, and responding to complex queries.

Prominent models like the GPT series [16] have demonstrated exceptional generalization capabilities, allowing them to be fine-tuned for domain-specific tasks. LLMs can generalize to different tasks by using what is called "in context learning", which is when they use information that is given in the prompt to better perform a desired task (i.e., when you use few shot learning)[9].

A key factor in tweaking LLM outputs is the temperature parameter [3], which influences the randomness of the model's predictions. Lower temperature values make the model more deterministic, prioritizing highly probable tokens during text generation and ensuring consistency in outputs. In contrast, higher temperatures introduce greater randomness, allowing the model to explore less probable tokens and generate more diverse and creative outputs. The choice of temperature directly affects the trade-off between precision and creativity in the model's responses.

2.3 Prompting

Prompting is a central technique in interacting with LLMs. Instead of training models from scratch for specific tasks, prompting allows models to better take advantage of pre-existing knowledge learnt during large-scale training [12]. Prompts are textual inputs designed to guide the model through the desired tasks, such as text generation, summarisation, translation or even logical reasoning [17]. In the context of this work, several prompting strategies were tested, beginning with Zero-shot prompting (ZS) [14], which involves providing only a task description without any examples. This approach tests the model's ability to execute the desired task based on its pre-trained knowledge alone, requiring it to infer the steps needed to carry out the task solely from the instructions provided. Few-shot prompting [13] involves including a small number of examples within the prompt to provide contextual guidance, enabling the model to better understand the structure and requirements of the task. Few shot prompting can be enhanced by selecting which examples are shown to the model in an intelligent manner, for example, via similarity to the task's input. In the context of rule description generation, for instance, showing the human generated description for a very similar rule can better inform the model on how the task is to be carried out. CoT, or Chain-of-Thought prompting, is a strategy where the model is encouraged to break down the task into smaller, sequential steps before arriving at the final output [20]. By including the phrase "Let's think step by step" in the prompt, the model is prompted to articulate intermediate steps in its reasoning, which often leads to more structured and precise outputs [4].

In this work, a Retrieval-Augmented Generation (RAG) [19] inspired approach was used to enhance the rule description and name generation process. RAG combines information retrieval with generative models, allowing LLMs to leverage external data to improve the relevance and accuracy of their outputs. Instead of solely relying on the model's pre-existing knowledge, RAG retrieves relevant information from a database or document set, which is then used to inform and guide the model's output. This approach enhances the model's ability to generate more accurate, contextually relevant, and factually grounded responses, particularly in tasks that require specialized knowledge or precise information.

2.4 ROUGE

ROUGE (Recall Oriented Understudy for Gisting Evaluation) [8] is a widely used metric for evaluating the quality of text generation by comparing machine-generated text with reference texts. It focuses on n-gram overlap, precision, and recall, offering valuable insights into textual similarity and relevance. In the context of this work, ROUGE was employed to assess the performance of various prompting strategies and the effectiveness of different models, providing a quantitative measure for comparing the generated rule descriptions and names against reference data. This evaluation helps to understand how well the models perform and produce contextually accurate outputs.

2.5 BERT

BERT (Bidirectional Encoder Representations from Transformers) [18] is a pre-trained model that generates embeddings, which are vectors representing text, capturing semantic and structural relationships. These embeddings are relevant to the work because they allow for more accurate comparison between rules by considering not only the words but also their semantic context. Using BERT-generated embeddings enables more efficient similarity measurement between rules, overcoming approaches based solely on word-based similarity, thus improving the accuracy in identifying relevant relationships.

3 Methodology

This study employs a quantitative research design to evaluate the ability of LLMs to generate natural language descriptions and names for rules derived from induction algorithms. First, the data curation process and the methodology for generating the ground truth names and descriptions used for the evaluation are detailed. Subsequently, the performance across different prompting techniques is benchmarked to identify the most effective strategies for achieving accurate and meaningful rule descriptions and names.

3.1 Data and Rule Induction

A publicly available dataset, Bank Account Fraud (BAF) [5], was chosen for this study. The dataset was generated by applying state-of-the-art tabular data generation techniques on an anonymized, real-world bank account opening fraud detection dataset, carrying a set of challenges that are commonplace in real-world applications, including temporal dynamics and significant class imbalance.

For this analysis, the "Base" variant of the BAF suite was chosen, which is a synthetic account opening fraud dataset with one million instances, a "month" column to allow temporal validation, and three protected attributes (age group, employment status, and % income) to allow for fair ML evaluation.

Rules were generated using RIFF, selected for its state-of-the-art performance and its effectiveness in building low False Positive Rate (FPR) rule sets. A total of 50 rules, derived using RIFF were subsequently reviewed, each of which was manually assigned a description and a name, to serve as the ground truth when evaluating the LLMs' outputs. It is also important to note that the dataset of 50 rules was not chosen randomly. Instead, it was carefully analyzed to ensure that the distributions of rule length, in terms of the number of conditions it contains, and variable presence in the full rule set were preserved in the sample. The rule dataset was then randomly split into a development and a testing set. The development set, composed of 20 rules, contains the instances that can be used for examples in methods like few-shot prompting or as a general source of information in other techniques such as intelligent few-shot selection. The testing set contains 30 rules to guarantee that the results are statistically significant, and encompasses the instances on which the LLMs' will be evaluated, when tasked with generating a description and a name for them.

3.2 Zero-Shot Prompting

As an initial step in the process of studying LLMs' ability to generate natural language descriptions and names for rules obtained by an induction algorithm, it is important to understand the effects of starting with the simplest prompting technique, zero-shot prompting. With this in mind, the prompt "Create a name and a description for this rule. Rule: ", followed by the test rule and the placeholders "Name:\n Description:", instructing the model to separate its response into distinct sections for easier retrieval, was designed to clearly specify the task and provide the model with the rule to process. This first examination can thus serve as a baseline for subsequent techniques.

Additionally, as a possible improvement, zero-shot CoT was also implemented, by adding the sentence "Let's think step by step." at the end of the prompt. End the text with 'Description:' and 'Name:' sections." . This modification could potentially lead the model to interpret the rule in its composing variable-condition pairs and also provides, once again, clear instructions so that the generated description and name can be more easily fetched.

3.3 Few-Shot Prompting

As an improvement upon zero-shot prompting, we aimed to understand the effects of gradually adding information to the prompt, through few-shot prompting with an increasing number of shots or examples. This procedure can thus serve as a foundation for comprehending what amount of information enhances the models' performances.

Furthermore, since the temperature parameter plays a crucial role in controlling the randomness of a language model's output, that parameter is also varied for each model.

Thus, in this study, the temperature of each model is varied from 0.25 to 1 in increments of 0.25 and, simultaneously, the number of shots was adjusted from a minimum of 1 to a maximum of 5. This variation makes it possible to observe how the models' behavior changes with additional information in the prompt and under different levels of randomness. By analyzing the outputs, the optimal temperature and number of shots for each model can be identified.

For the few-shot prompting method, used in the dual variation described above, the prompt was built by adding each example in the format "Rule: \n Description: \n Name:", where each of these indicators is followed by its respective part of the example and then finishing with "Rule:", followed by the test rule. This structure provides clear examples in terms of the desired style of description and name and also expected structure of the response. Few-shot CoT was also

implemented, by adding "Let's think step by step. End the output with 'Description:' and 'Name:' sections." at the end of the prompt, once again, trying to lead the model to better understand the rule it has to describe.

3.4 One-Shot Prompting based on Similarity

Following the investigation into the optimal number of shots for prompting, one-shot prompting was further explored. The goal was to create a method inspired by RAG that allowed the model to, when prompted to generate a description and a name for a certain rule, retrieve the most similar rule in the development set to serve as the best possible example in order to maximize its performance. The underlying assumption is that the most similar rule will share key variables with the test rule, making it an ideal reference for guiding the LLM to generate desirable descriptions and names.

This approach is loosely based on RAG, which combines retrieval-based methods with generative models. In RAG, the model retrieves relevant information from an external source (i.e. a database) to inform its response. Similarly, in this method, the model retrieves the most relevant rule from the development set to help generate the best possible output.

The main rationale for using one-shot prompting lies in addressing the exploration vs. exploitation dilemma. Providing multiple similar examples in an n-shot prompting setup could narrow the model's perspective, potentially overlooking important variables and restricting its creativity. By offering just one similar rule, we assume that this approach allows the model to generalize and explore a wider range of possibilities while maintaining a degree of focus. Additionally, one-shot prompting was chosen for its simplicity, computational efficiency, and the reduced cost of API queries when providing only a single example.

In terms of the prompt used, the exact same structures as in few-shot and few-shot CoT prompting were used, only with the particularity of adding one single example.

To quantify similarity, three distinct measures were developed and employed. These measures are detailed in pseudocode in the appendix (1, 2, and 3, respectively).

The first similarity measure leverages the structured nature of the rules, which consist of conjunctions of inequalities in a consistent variable-sign-number format (see the example in section 1). To calculate the similarity between a test rule and a rule from the development set, the algorithm evaluates each inequality in the test rule. For each inequality, it checks whether the variable is present in the training rule. If the variable is not found, a score of 0 is assigned. If the variable is present but the signs differ, a score of 0.5 is assigned. If both the variable and the sign match, the score is 1. The algorithm iterates through all inequalities in the test rule, summing the scores to produce the final similarity measure.

The second similarity measure aims to improve upon the first, adding importance to the order in which the variables appear in the rules. This is done by adding an extra value to the score of each inequality, equal to $o_f/(1+i)$, where o_f is the order factor, a value that indicates the level of importance attributed to the order of the variables in the test rule and i represents the index distance, i.e., the absolute value of the difference between the position of a variable in the test rule and of that same variable in the development rule. This added term implies that the most similar rule will be the one not only where the same variables and signs appear, but also the one where they are placed in the most similar order to the test rule, meaning that the example description and name will follow a structure that is even closer to the desired output.

The third similarity measure employs a more advanced approach by leveraging BERT, a pre-trained transformer model, to generate embeddings for both the test rule and each rule in the development set. In this context, an embedding is a numerical representation of the rule, capturing its semantic meaning and structure in a high-dimensional vector space. To compute the similarity between the test rule and a training rule, their respective embeddings are compared using cosine similarity, which measures the cosine of the angle between the two vectors, with a value closer to 1 indicating greater similarity. By using BERT-generated embeddings, this method aims to account for deeper semantic relationships and structural nuances within the rules, potentially providing a more accurate similarity measure than methods based purely on rule components.

3.5 Manual Rule Checking Metric

At last, after studying and developing prompting techniques, the analysis shifted to the outputs of the models, focusing on evaluating their adherence to the variables included in the test rule. This evaluation aimed to determine whether the generated output descriptions included all the variables present in the test rule and whether they introduced any extraneous variables not present in the original input, but included in the dataset. This process provided insights into the completeness and precision of the model's responses.

To systematically analyze the variables in the outputs, a framework was developed to identify the presence of each possible variable, even if described by synonyms. This was achieved by defining specific keywords and rules associated with each variable. For example, for the variable *keep_alive_session*, the corresponding rule ("session" AND ("alive" OR "active" OR "running" OR "open" OR "uninterrupted" OR "ongoing" OR "sustained" OR "without interruption" OR "no interruption")) was used to confirm its inclusion in the output.

Using this framework, the first step was to check whether the output descriptions included the variables defined by the rules. Variables that met the conditions specified by the rules were considered present in the output. The next step involved comparing the variables identified in the output descriptions with those in the original test rule. Variables included in the description but not in the test rule were

flagged as extraneous, while any variables from the test rule that were absent from the output were marked as missing. This comparison served as a metric to evaluate the accuracy and completeness of the model's descriptions.

Finally, as a refinement step and for easier comparison with other results, Recall and Precision metrics were derived from this information. Recall was defined as the ratio between the number of correct variables found in the description and the total number of variables in the rule, while Precision was established as the number of correct variables found in the description over the total number of variables in the description. By developing this metric, it will be possible to compare the rankings of the various prompting techniques when evaluating the results with ROUGE or this Manual Metric.

4 Results

In this section, the results of the several experiments performed are shown and discussed.

To obtain a broad view of the effect of different prompting techniques on the output of LLMs, three different OpenAI models were chosen to perform the tests: GPT-4 Turbo, GPT-4o and GPT-4o mini. These models were selected for their varying computational capabilities, enabling a comparison of performance across scales and configurations.

The metrics chosen for analysis were ROUGE-1 Recall and F1, with the former focusing on the proportion of relevant words from the reference text that were successfully captured by the model's output, and the latter representing the harmonic mean of precision and recall, balancing the trade-off between capturing relevant information and avoiding irrelevant or excessive content. Notably, F1 penalizes overly long answers by incorporating precision, ensuring that higher scores are achieved only when the output is both relevant and concise. Together, they complement each other for a holistic view of the results.

4.1 Data Distribution

As described at the end of the subsection 3.1, the 50 rules in the dataset for this study were sampled in order to maintain the same distributions of rule length, in terms of the number of conditions they contain, and variable presence as in the full rule set, which contained a total of 606 rules.

Thus, Figures 1 and 2 illustrate the distribution of rule lengths in the full rule set, and the 50 sampled rules, respectively. Notably, while the 50-rule sample lacks rules with very low (1-2) or very high (11-12) numbers of conditions, it effectively captures the overall shape and balance of the distribution across mid-range length (3-10), particularly the peak around 6-7 terms. This suggests that the sampled subset successfully retains the key characteristics of the full rule set's distribution in terms of length.

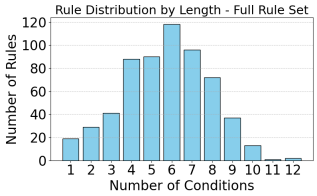

Figure 1. Distribution of rules' length in the full rule set.

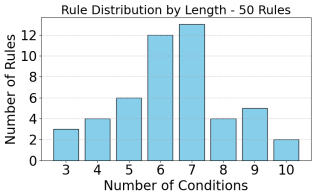

Figure 2. Distribution of rules' length in the 50 sampled rules.

Moreover, in the Appendix, Figures 18 and 19 present the distribution of each variable's presence in the full rule set and the 50 sampled rules, respectively. Despite the smaller scale, the sample closely mirrors the key trends of the full set of rules, as the variables most frequently used, e.g. *housing_status_BA*, *device_os_windows*, and *name_email_similarity*, dominate both distributions, while the less frequent variables maintain similar relative proportions. This alignment demonstrates that the sampled subset effectively preserves the variable usage patterns of the complete rule set.

4.2 Zero-Shot Prompting

Following the methodology described in subsection 3.2, the results obtained for the three LLMs using zero shot prompting are represented below:

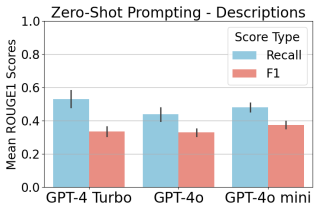

Figure 3. Average ROUGE-1 Recall and F1 scores for descriptions, using zero-shot prompting.

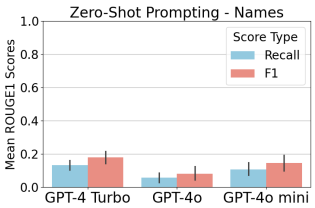

Figure 4. Average ROUGE-1 Recall and F1 scores for names, using zero-shot prompting.

Firstly, comparing the results obtained for descriptions and names, it is clear that the scores for names are significantly lower than those for descriptions across all models. This discrepancy is expected given the nature of the tasks. Descriptions tend to be more detailed and explanatory, which allows the model more flexibility to generate a variety of acceptable outputs that capture the conditions within the rule. In contrast, names are more compact and must encapsulate the key elements of the rule succinctly, leading to a narrower set of acceptable outputs and, consequently, lower scores in both Recall and F1 metrics. Additionally, the expected number of n-grams in a name is significantly smaller, meaning that even a small change can have a larger impact on the

result compared to descriptions. For instance, consider the following example:

Reference Name: "BA Housing without Windows, high credit limit, not keeping session alive, and limited name-email similarity"

Generated Name: "Selective Eligibility for Credit Enhancement"

As mentioned, the reference name is detailed and context-specific, making it clear and informative, whereas the generated name is too broad and fails to encapsulate the rule's key attributes, resulting in lower performance metrics for name generation tasks.

Focusing on descriptions (figure 3), GPT-4 Turbo achieves the highest Recall score, close to 0.6, indicating that it is more capable of capturing a higher proportion of relevant n-grams from the reference text. However, its F1 score is slightly above 0.35, showing that while it captures many relevant details, it may also produce excessive content, affecting its precision.

GPT-4o-mini outperforms GPT-4 Turbo in terms of F1 score, despite having a lower Recall score. Its F1 score is slightly below 0.4, while its Recall is a bit over 0.4. This suggests that GPT-4o-mini is more balanced in its approach, achieving a better trade-off between precision and recall. It indicates that while it may not capture as many relevant details as GPT-4 Turbo, the details it does capture are more precise, leading to a more concise output. GPT-4o, on the other hand, is outperformed by GPT-4 Turbo and GPT-4o-mini in terms of both metrics.

In the case of names (figure 4), all models demonstrate lower performance across the board, demonstrating the struggle with the concise nature of the naming task.

These results highlight that zero-shot prompting does not deliver strong results, and the nature of the task significantly influences its performance.

4.2.1 Zero-Shot CoT Prompting.

In Figures 5 and 6, the results of Zero-Shot CoT Prompting for descriptions and names are presented, respectively.

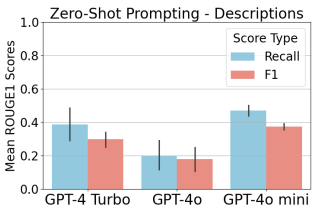

Figure 5. Average ROUGE-1 Recall and F1 scores for descriptions, using zero-shot CoT prompting.

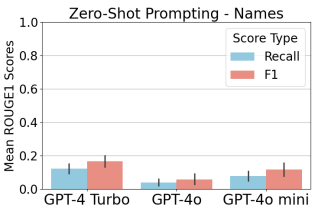

Figure 6. Average ROUGE-1 Recall and F1 scores for names, using zero-shot CoT prompting.

When comparing the results of Zero-Shot CoT prompting with traditional Zero-Shot prompting, a general decline in performance can be observed. This decrease is evident in both descriptions and names. This decline is likely due to a shift in how the task is approached with the addition of the CoT methodology.

After testing various prompting strategies, the most effective prompts were identified as those that framed the task as filling in the missing information. When the phrase "let's think step by step" was added to the end of the prompt, as suggested by the CoT method, the model's focus shifted.

This addition breaks the completion-oriented thought process, leading the model to deviate from simply completing the given information. As a result, the model becomes less effective.

4.3 Few-Shot Prompting

Following the procedure described in subsection 3.3, the results of simultaneosuly varying the number of shots and the temperature of each model were obtained.

Figures 7 and 8 present the averaged ROUGE-1 recall scores across all test rules for different numbers of shots and temperatures, based on the descriptions and names generated by GPT-4 Turbo, respectively.

It is possible to conclude that the introduction of a single example both improves the results and reduces the variability of the answers, noticeable by the smaller 95% confidence intervals. Moreover, from one to two examples, the recall continues to improve for descriptions and names, but when moving onto three and more shots, the results do not vary substantially. This plateau in the plots means that the greater computational expense of adding more than two examples does not compensate in terms of the output provided. By following this rationale, the ideal number of shots for GPT-4 Turbo is established as 2.

Regarding the temperature, in the descriptions plot, the 4 lines are very close to each other, implying that changing the randomness of the model's output does not produce a great change in its results. On the other hand, in the plot associated with the generated names, it is observed that for a low number of shots (1-3), a temperature of 0.25 yields the highest average recall. While the other temperature values produce similar average recalls, the error bars overlap, making it difficult to definitively conclude that 0.25 is the optimal choice. Nevertheless, due to its superior average performance, the optimal value of temperature for the GPT-4 Turbo model was thus fixed at 0.25.

For the remaining two models, GPT-4o and GPT-4o mini, the same analysis procedure was followed and their respective average ROUGE-1 recall scores for generated descriptions and names are represented in the appendix by Figures 20, 21, 22 and 23. The results across models were very much alike, which made it possible to use the same rationale in choosing the optimal number of shots and value of temperature for each of the models. Thus, for both GPT-4o and GPT-4o mini, the ideal number of examples was also defined

as 2, due to its favorable balance between computational expense and results, and the optimal temperature was set at 1, which introduces the highest level of randomness. However, the results across different temperatures were very close, indicating that the choice of temperature does not significantly affect the outcomes.

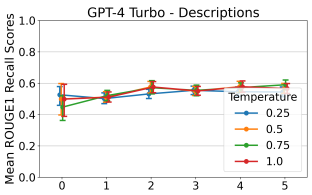

Figure 7. Average ROUGE-1 recall scores for descriptions by GPT-4 Turbo across shots and temperatures.

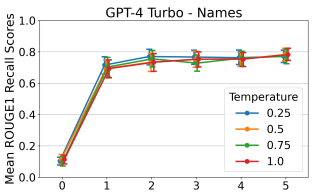

Figure 8. Average ROUGE-1 recall scores for names by GPT-4 Turbo across shots and temperatures.

Having now established the ideal conditions for each model to apply two-shot prompting, it is now possible to observe the results produced with this prompting technique.

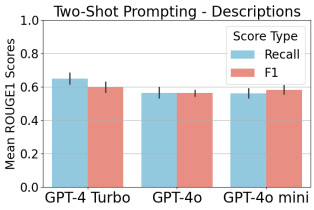

Figure 9. Average ROUGE-1 Recall and F1 scores for descriptions, using two-shot prompting.

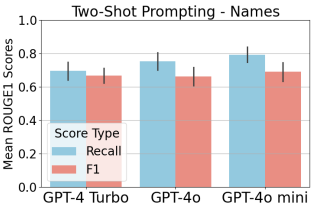

Figure 10. Average ROUGE-1 Recall and F1 scores for names, using two-shot prompting.

Figures 9 and 10 illustrate the metrics obtained for each model when evaluating the generated descriptions and names, respectively. A key comparison can be drawn with the zero-shot prompting results. Introducing two examples significantly improved performance, since for descriptions, recall and F1 values rose from approximately 0.5 and 0.3, respectively, to 0.6, while for names, recall and F1 scores increased dramatically, from below 0.2 to around 0.7.

The increase in description scores can be attributed to the shots, which provided clear guidance on commenting on variables in the test rule and structuring descriptions, particularly addressing each condition. In the case of names, the improvement may be attributed to the fact that reference names focus only on key variables in the rule. The examples help the model identify and fixate on these critical variables and how to describe them, which implies that, when the examples are closely aligned with the test rule, most of the generated name can be copied directly from the prompt, requiring only minor adjustments to reflect the specific conditions. For example, when using as examples the names "BA Housing, non-Windows device, low name-email similarity, invalid home phone, long-term address and AC payment type" and "BB housing with Windows device, no other cards, credit risk score and limited bank applications", GPT-4o mini generated the name "BA Housing, Windows device, no other cards, low name-email similarity, non-active session, low intended balance, multiple distinct emails, invalid home phone, and acceptable credit risk score.", where we can confirm that the majority of the output is extracted directly from the examples. Descriptions, on the other hand, go through each condition in the test rule, describing in many different ways the same variables or type of inequality, which introduces much more variability in the possible output, explaining why the increase for this task is not so noticeable.

Regarding the comparison across models, all three produced very similar results overall. However, GPT-4 Turbo achieved slightly higher Recall and F1 scores for descriptions, while GPT-4o and 4o mini outperformed it in metrics for names. This suggests that the 4o models may be better tuned to identifying critical portions of the examples that can be directly applied to the answer and, in contrast, the Turbo version appears to be more inclined toward extracting information in a broader, more generalized manner.

Furthermore, in terms of the metrics themselves, the close similarity between Recall and F1 across the three models for the descriptions suggests that the models are being able to capture the relevant information effectively, as indicated by the Recall, and, at an almost identical level, also generate outputs that are well-aligned with the reference texts in terms of structure and phrasing, as shown by the F1. This balance indicates that the models perform consistently in extracting and presenting information without over-generating irrelevant or unnecessary details. For the names, in the three cases, the Recall is slightly greater than the F1, meaning that while the models are effective at identifying and including relevant elements from the references (high Recall), the overall precision of their outputs is slightly lower. This suggests that the generated names may contain some less precise information, leading to a slight imbalance between capturing relevant content and maintaining alignment with the ideal reference names.

Additionally, experiments were also made by employing few-shot CoT prompting. The results obtained for descriptions and names are shown in figures 11 and 12, respectively.

The results produced by employing this prompting technique follow a similar structure to those without CoT, in the sense that the three models performed better for the names than for the descriptions and, once again GPT-4 Turbo was the best model in the descriptions, while being outperformed by GPT-4o and 4o mini in the names, two models that had very close results between themselves for the two generation tasks.

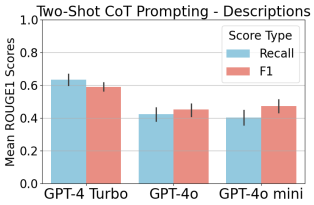

Figure 11. Average ROUGE-1 Recall and F1 scores for descriptions, using few-shot CoT prompting.

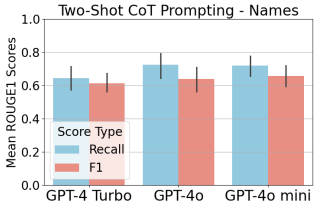

Figure 12. Average ROUGE-1 Recall and F1 scores for names, using few-shot CoT prompting.

Taking a closer look, the metrics for the names showed a slight reduction, while for descriptions, the Recall and F1 scores of GPT-4o and 4o-mini experienced a more noticeable decline. This general drop supports the idea mentioned in 4.2.1 that adding detailed suggestions (e.g., "Let's think step by step") and specific instructions regarding the answer format shifts the model's interpretation of the task. Instead of viewing it as a simple completion exercise, the model is prompted to adopt a more complex approach, a change that could discourage it from closely imitating the provided examples, ultimately hindering its performance. This technique was also expected to not perform so well, since it was developed and is usually used in numerical tasks, not linguistic ones [20]. It is important to note, however, how GPT-4 Turbo is more resilient to the modification, perhaps not attributing so much importance to the end of the prompt.

4.4 One-Shot Prompting based on Similarity

As detailed in Section 3.4, the subsequent step involves refining the prompt by selecting the training rule most similar to the test rule, taking in consideration three distinct similarity measures, to assess and identify the rule that would serve as the most relevant and representative example. The decision to apply these similarity measures within a one-shot prompting framework was made based on the results from Section 4.3.

Following the analysis shown in Figures 7 and 8, we observed that the improvement from zero shots to one shot is significantly greater than the improvement from one shot to two shots. This indicates that introducing just a single, random example leads to a considerable performance boost. Beyond two examples, the improvements plateau, indicating that adding more shots does not substantially enhance the output quality relative to the computational cost. Given that even a random single example yields substantial gains, refining the selection of this example based on similarity could potentially match or surpass the performance achieved with two or more shots, while being less computationally expensive and simpler to implement. This conclusion aligns with section 3.4 where one-shot prompting was selected due to its simplicity, computational efficiency, and its balance between exploration and exploitation.

Since the main focus in this section is the effect produced by each one of the three different algorithms designed to choose the best example for a certain test rule based on similarity, the results will now be aggregated by model and only F1 will be shown, since it includes Recall in its harmonic mean and representing both metrics would clutter the plots. Thus, figures 13, 14 and 15 present, respectively, the average F1 obtained for names and descriptions when applying the three similarity algorithms to GPT-4 Turbo, GPT-4o and GPT-4o mini. Additionally, the results for two-shot prompting are also included to facilitate the comparison.

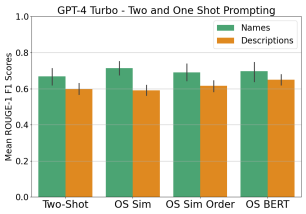

Figure 13. Average F1 scores for three types of one-shot prompting, using GPT-4 Turbo.

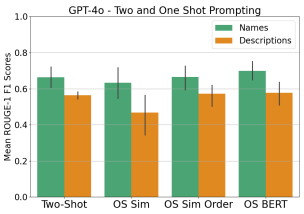

Figure 14. Average F1 scores for three types of one-shot prompting, using GPT-4o.

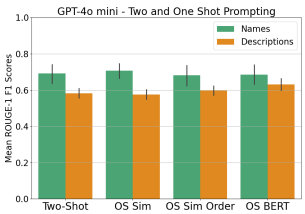

Figure 15. Average F1 scores for three types of one-shot prompting, using GPT-4o mini.

The results in these images demonstrate that, across all three models, the average F1 scores for both names and descriptions achieved by our one-shot prompting algorithms successfully match or exceed the performance of two-shot prompting, confirming their validity and robustness.

The first algorithm, One-Shot Similarity (OS Sim), evaluates similarity by matching variables and signs between the test and training rules. It achieves strong results by leveraging the structured nature of the rules, ensuring that selected examples align closely with the test rule's variables and condition signs. While OS Sim provides a solid baseline for similarity-based selection, it does not account for variable order, which limits its ability to model structural nuances. Despite this, it performs comparably or even better to two-shot prompting across most settings, particularly for names.

Building on the first similarity measure, the second algorithm, One-Shot Similarity Order (OS Sim Order), incorporates the order of variables into the similarity calculation, assigning additional weight to examples where the variable ordering aligns more closely with the test rule. The slightly improved performance of OS Sim Order, particularly for descriptions, possibly highlights the importance of structure and ordering in generating outputs that more accurately reflect the desired rule format, since the example descriptions and names will follow a structure that may be more directly imitated than those obtained with OS Sim, even if sharing almost the same variables and conditions. This suggests that order-aware example selection leads, for certain cases, to a closer match between the example's descriptive form and the target rule.

The last technique, One-Shot BERT (OS BERT), which leverages BERT embeddings to capture semantic and structural information, represents a more advanced approach to similarity. However, it is important to note that the rules in this study are highly structured, following a consistent format. This uniformity in structure means that the rules do not inherently exhibit deep semantic relationships. Instead, their similarity is primarily determined by the presence and arrangement of variables, their associated signs, and thresholds. Despite this, the OS BERT algorithm performs strongly across all models and tasks, suggesting that BERT embeddings are able to capture nuanced structural patterns and latent relationships between rules, even in the absence of significant semantic variation. For instance, embeddings might effectively encode the co-occurrence or typical usage patterns of specific variables and signs, offering a more holistic representation of rule similarity than simpler, component-based methods. Thus, while the structured nature of the rules minimizes the role of semantics in the traditional sense, the results validate the utility of embedding-based approaches like OS BERT for identifying relevant examples in highly constrained, structured domains, highlighting the flexibility and strength of these embeddings in capturing both explicit and implicit patterns, even in scenarios where the input data is rigidly formatted.

The results demonstrate that one-shot prompting based on similarity is a highly effective strategy for generating high-quality outputs, with all three similarity measures showing strong performance across both names and descriptions. While variations exist in which approach performs best, each method offers unique advantages. For descriptions, these methods excel by leveraging richer context and structural alignment to preserve the semantic intent and format of the test rule. For names, the inclusion of examples containing the same key variables as the test rule enhances performance, highlighting the versatility and effectiveness of these algorithms in capturing nuanced patterns. It is also curious how OS BERT, probably due to its ability to capture subtle patterns in rule structure even within their rigid and formulaic nature, leads, in some cases, to examples that closely align with the test rule and more accurate and contextually appropriate outputs.

A main conclusion is that targeted example selection using similarity measures, whether based on explicit variable matching, order sensitivity, or embeddings, offers significant advantages over relying on more but randomly chosen examples. By focusing on relevance, these methods reduce noise and enhance the model's ability to generalize from fewer examples, which can be particularly impactful in scenarios where the input-output mappings are highly structured, as it ensures that the examples chosen reinforce the specific patterns needed to generate correct outputs. Overall, these findings validate the effectiveness of similarity-based one-shot prompting as an efficient and reliable approach, with the potential to outperform approaches that rely on larger but less precise datasets.

For completeness of results, One-Shot CoT prompting was also tested and its results are included in the appendix (figures **??**, **??** and **??**), but do not seem important to comment since once again, as with zero and two-shot prompting, produced lower results than its non CoT counterpart.

4.5 Manual Rule Checking Metric

As described in 3.5, a Manual Metric that measures rule completeness and extraneous variables was built, allowing Recall and Precision metrics to be derived from it. Now, since the main focus of this analysis is to compare how the ranking of the different prompting techniques changes when evaluating the results with ROUGE or this Manual Metric, it is best to only focus on one model. Thus, GPT-4o was chosen since it serves as OpenAI's default model. With this in mind, figures 16 and 17 present, respectively, the ROUGE-1 and Manual Metric average Recall and F1 scores obtained for the generated descriptions, across all tested prompting types, not considering CoT variations, since they generally result in a decline of the results.

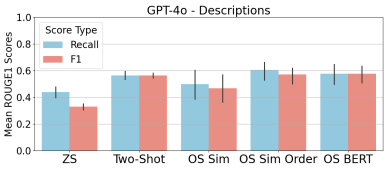

Figure 16. ROUGE-1 results across prompting techniques for GPT-4o.

Based on the two plots provided, the rankings of the prompting techniques by the ROUGE-1 metric and the Manual Metric share some similarities, but also differ significantly in some aspects, indicating a shift in how performance is measured when moving from a lexical-based to a semantics-oriented evaluation.

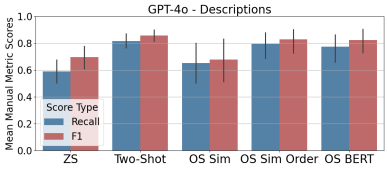

Figure 17. Manual Metric results across prompting techniques for GPT-4o.

In Figure 16 (ROUGE-1), "OS Sim Order" emerges as the best performing technique, closely followed by "OS BERT" and "Two-Shot" prompting, then comes "OS Sim", with a slight decrease in performance and, finally, "ZS" at the last place. On the other hand, in figure 17 (Manual Metric), "Two-Shot" was the best prompting technique, with "OS BERT" and "OS Sim Order" next in line, followed by "OS Sim" and "ZS", which have very similar results.

Thus, in both cases, the three best-performing techniques were two-shot prompting, one-shot considering order, and one-shot leveraging BERT. However, it is notable how two-shot prompting rises from third place with ROUGE-1 to the best-performing technique when evaluated with the Manual Metric, while, contrarily, "OS Sim Order" and "OS Sim," which ranked high under ROUGE-1, show a slight decline in rank. This shift may indicate that by providing more examples in the prompt, the model possibly gains access to a broader range of variations on how to build names and descriptions or how to comment each variable, implying that, when applying two-shot prompting, the model performs the generation tasks more freely, i.e., not imitating directly the examples, in comparison with one-shot prompting. This idea is even more sustained by remembering that "Two-Shot" uses two random examples, while the OS methods choose the most similar example to the test rule, really leading the model to imitate it. Since ROUGE is a limited metric that cannot account for synonyms or paraphrases, it penalizes Two-Shot prompting, as this method tends to abstract more and diverge from the reference text. In contrast, the Manual Metric, which can detect variable inclusion regardless of lexical variation, rewards Two-Shot prompting for its ability to capture the necessary variables, even when using alternative phrasing. This highlights the strength of Two-Shot prompting in generating content that aligns with the underlying rules, despite lexical differences.

Moving onto the remaining two techniques, placed at the bottom of the ranking, there are also changes that need to be remarked regarding the evaluation with both metrics of zero shot prompting and one-shot based on the first similarity algorithm. For ROUGE-1, zero-shot prompting was clearly below in performance to "OS Sim", however, when looking at the evaluations with the Manual Metric, they present very close results, suggesting that while "ZS" prompting lexical similarity may be lower, it captures a comparable proportion of relevant variables or aligns better semantically. This variation can be explained by the fact that in zero-shot prompting the model is not informed about the desired structure of a description, leading it to generate text far from the description, but seemingly including a good percentage of the variables in the test rule.

These disparities highlight the limitations of ROUGE-1 in evaluating the semantic fidelity of generated descriptions and underscores the importance of using metrics like the Manual Metric to capture deeper semantic nuances.

5 Conclusions

Finally, having reached the final stage of this study, it is possible to draw meaningful conclusions about the effectiveness of large language models in rule synthesis and evaluation.

Firstly, it was made clear that the application of large language models (LLMs) to generate names and descriptions that synthesize rules demonstrates significant potential in improving the transparency and communication of those rules. By creating clear and concise representations, these techniques can help explain complex decisions to clients, stakeholders, and other non-technical audiences, fostering trust and understanding.

Next, in the several steps of prompt-engineering experiments it was revealed that one-shot prompting techniques based on similarity achieve results comparable to, or even surpassing, few-shot prompting, with the added advantage of reduced computational cost. This challenges the assumption that more examples always lead to better performance, highlighting the importance of carefully curated examples that align closely with the test rule.

Following this, the newly designed custom metric highlighted the complementarity between ROUGE-1 and the Manual Rule Checking Metric. While ROUGE-1 remains valuable for measuring lexical overlap with reference texts, the Manual Metric provided deeper insights into the model's ability to handle paraphrases and synonyms. Together, these metrics provide a more holistic perspective on the quality of the generated outputs and the effectiveness of the prompting strategies.

In conclusion, the combination of similarity-based one-shot prompting and multi-faceted evaluation metrics paves the way for more efficient and insightful applications of LLMs in rule-based systems. Future work could explore integrating these techniques into practical workflows, further enhancing interpretability and usability in real-world decision-making contexts.

References

[1] Niki Parmar Jakob Uszkoreit Aidan N. Gomez Łukasz Kaiser Illia Polosukhin Ashish Vaswani, Noam Shazeer. [n. d.]. *Attention Is All You Need.* https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[2] Nitin Indurkhya e Fred J. Damerau. 2010. *HANDBOOK OF NATURAL LANGUAGE PROCESSING* (2nd. ed.). Chapman Hall/CRC, Cambridge, UK. https://karczmarczuk.users.greyc.fr/TEACH/TAL/Doc/Handbook%20Of%20Natural%20Language%20Processing,%20Second%20Edition%20Chapman%20&%20Hall%20Crc%20Machine%20Learning%20&%20Pattern%20Recognition%202010.pdf

[3] Matthew Renze; Erhan Guven. 2024. *The Effect of Sampling Temperature on Problem Solving in Large Language Models.* Retrieved Nov, 2024 from https://aclanthology.org/2024.findings-emnlp.432.pdf

[4] Takeshi Kojima; Shixiang Shane Gu; Machel Reid; Yutaka Matsuo; Yusuke Iwasawa. 2022. *Large Language Models are Zero-Shot Reasoners.* Retrieved May 24, 2022 from https://arxiv.org/abs/2205.11916

[5] S. Jesus, J. Pombal, D. Alves, A. Cruz, P. Saleiro, R. P. Ribeiro, J. Gama, and P. Bizarro. 2022. Turning the Tables: Biased, Imbalanced, Dynamic Tabular Datasets for ML Evaluation. In *Advances in Neural Information Processing Systems.*

[6] Veenu Rani1; Syed Tufael Nabi1; Munish Kumar1; Ajay Mittal2; Krishan Kumar2. 2022. *Self-supervised Learning: A Succinct Review.* Retrieved December 26, 2024 from https://link.springer.com/article/10.1007/s11831-023-09884-2

[7] Xuanfan Ni; Piji Li. 2024. *A Systematic Evaluation of Large Language Models for Natural Language Generation Tasks.* Retrieved May 16, 2024 from https://arxiv.org/abs/2405.10251?utm_source=chatgpt.com

[8] Chin-Yew Lin. [n. d.]. *ROUGE: A Package for Automatic Evaluation of Summaries.* https://aclanthology.org/W04-1013.pdf

[9] Chin-Yew Lin. 2024. *Language Models are Few-Shot Learners.* Retrieved May 28, 2020 from https://arxiv.org/abs/2005.14165

[10] Chin-Yew Lin. 2024. *Looking for a few good metrics: ROUGE and its evaluation.* Retrieved June 2, 2024 from https://research.nii.ac.jp/ntcir/ntcir-ws4/NTCIR4-WN/OPEN/OPENSUB_Chin-Yew_Lin.pdf

[11] Ana Sofia Gomes Carlos Soares Lucas Martins, João Bravo and Pedro Bizarro. 2024. *RIFF: Inducing Rules for Fraud Detection from Decision Trees.* Retrieved August 23, 2024 from https://arxiv.org/pdf/2408.12989

[12] Jonathan May. 2024. *Parameter Efficiency, Few-Shot, Zero-Shot, Prompting.* Retrieved September 29, 2024 from https://jonmay.github.io/USC-CS662/assets/files/llm.pdf

[13] Samuel Kwok; James Morice. [n. d.]. *Few-Shot Prompt-Tuning: An Extension to a Finetuning Alternative.* https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/final-projects/JamesJMoriceSamuelEdwardKwok.pdf

[14] Zijun Wu1; Yongkang Wu2; Lili Mou. [n. d.]. *ZERO-SHOT CONTINUOUS PROMPT TRANSFER: GEN- ERALIZING TASK SEMANTICS ACROSS LANGUAGE MODELS.* https://arxiv.org/pdf/2310.01691

[15] Humza Naveed. 2024. *A Comprehensive Overview of Large Language Models.* Retrieved October 17, 2024 from https://arxiv.org/pdf/2307.06435

[16] OpenAI. [n. d.]. *GPT-4 Technical Report.* https://cdn.openai.com/papers/gpt-4.pdf

[17] Hemil Patel; Shivam Parmar. [n. d.]. *Prompt Engineering For Large Language Model.* https://www.researchgate.net/publication/379048840_Prompt_Engineering_For_Large_Language_Model

[18] Jacob Devlin; Ming-Wei Chang; Kenton Lee; Kristina Toutanova. 2024. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* Retrieved June 7, 2019 from https://aclanthology.org/N19-1423.pdf

[19] Yunfan Gaoa; Yun Xiongb; Xinyu Gaob; Kangxiang Jiab; Jinliu Panb; Yuxi Bic; Yi Daia; Jiawei Suna; Meng Wangc and Haofen Wang. 2024. *Retrieval-Augmented Generation for Large Language Models: A Survey.* Retrieved March 27, 2024 from https://arxiv.org/pdf/2312.10997

[20] Jason Wei; Xuezhi Wang; Dale Schuurmans; Maarten Bosma; Brian Ichter; Fei Xia; Ed Chi; Quoc Le; Denny Zhou. 2022. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* Retrieved January 28, 2022 from https://arxiv.org/abs/2201.11903

6 Appendix

Algorithm 1: Structured Similarity Measure Algorithm

```
Input: TestRule (list of inequalities), TrainingRule
       (list of inequalities)
Output: SimilarityScore (a numerical score)
Initialize SimilarityScore = 0;
foreach inequality in TestRule do
    Extract variable and sign from the inequality;
    if variable not in TrainingRule then
        Add 0 to SimilarityScore;
    else
        Find the corresponding inequality in
          TrainingRule;
        if signs are different then
            Add 0.5 to SimilarityScore;
        else
            Add 1 to SimilarityScore;
        end
    end
end
return SimilarityScore;
```

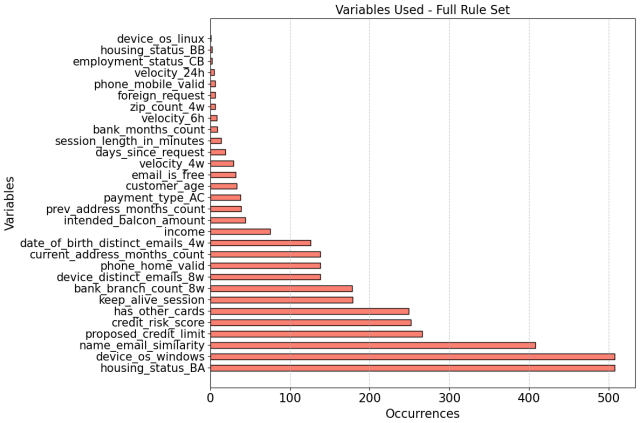

Figure 18. Distibution of Variable Presence in the Full Rule Set.

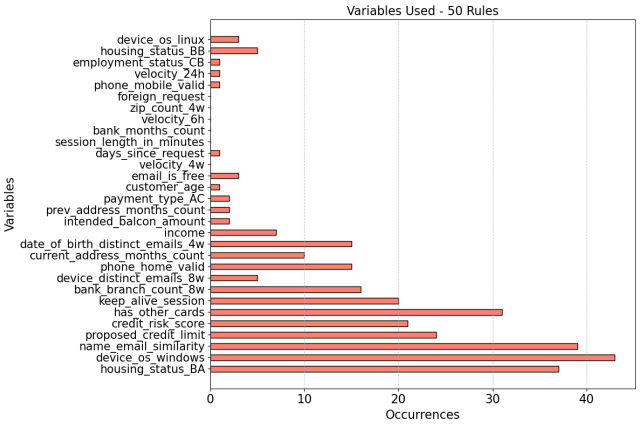

Figure 19. Distibution of Variable Presence in the 50 sampled rules.

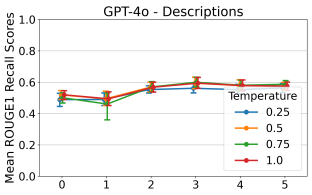

Figure 20. Average ROUGE-1 recall scores for descriptions by GPT-4o across shots and temperatures.

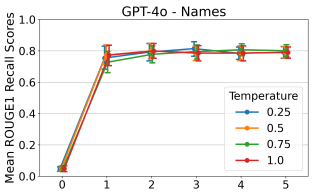

Figure 21. Average ROUGE-1 recall scores for names by GPT-4o across shots and temperatures.

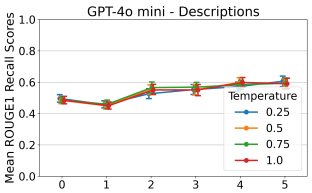

Figure 22. Average ROUGE-1 recall scores for descriptions by GPT-4o mini across shots and temperatures.

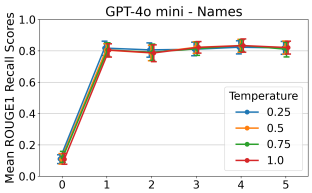

Figure 23. Average ROUGE-1 recall scores for names by GPT-4o mini across shots and temperatures.

Algorithm 2: Order-Aware Similarity Measure Algorithm

Input: TestRule (list of inequalities), TrainingRule (list of inequalities), OrderFactor (default = 1.0)
Output: SimilarityScore (a numerical score)

```
Initialize SimilarityScore = 0;
foreach inequality in TestRule do
    Extract variable and sign from the inequality;
    if variable not in TrainingRule then
        Add 0 to SimilarityScore;
    else
        Find the corresponding inequality in TrainingRule;
        if signs are different then
            Add 0.5 to SimilarityScore;
        else
            Add 1 to SimilarityScore;
        end
        Compute IndexDistance = |position of variable in TestRule − position in TrainingRule|;
        Compute OrderBonus = OrderFactor / (1 + IndexDistance);
        Update SimilarityScore += OrderBonus;
    end
end
return SimilarityScore;
```

Algorithm 3: BERT-Based Similarity Measure Algorithm

Input: TestRule (a textual representation of a rule), TrainingRule (a textual representation of a rule), BERT Tokenizer, BERT Model
Output: SimilarityScore (a numerical score)

```
Use the BERT Tokenizer to encode TestRule into a high-dimensional embedding using the BERT Model;
Use the BERT Tokenizer to encode TrainingRule into a high-dimensional embedding using the BERT Model;
Compute the cosine similarity between the embeddings of TestRule and TrainingRule;
return the computed cosine similarity as SimilarityScore;
```